Autonomous Robot Navigation in a Restaurant Environment

Comparison of Classical Path Planning and Reinforcement Learning
TurtleBot3 in Gazebo / ROS

Robotics Project Report

March 2026

GitHub Repository: `https://github.com/ValouPipop/Robotique`

Abstract

This report presents the design and implementation of an autonomous navigation system for a TurtleBot3 robot in a simulated restaurant. The robot must deliver orders from the kitchen to five tables while avoiding obstacles. Two paradigms are compared: **classical path planning** (A*, Dijkstra, Greedy + DWA controller) and **reinforcement learning** (PPO Actor-Critic). The comparison evaluates path efficiency, success rate, obstacle avoidance, and computation time.

Contents

1 Introduction **3**
1.1 Use Case . . . 3
1.2 Objectives . . . 3
1.3 System Overview . . . 3

2 Environment Setup **3**
2.1 Restaurant World . . . 3
2.2 Map Generation . . . 3
2.3 Coordinate Transformation . . . 4
2.4 Obstacle Inflation . . . 4

3 Classical Navigation **4**
3.1 Path Planning Algorithms . . . 4
3.1.1 A* Algorithm . . . 4
3.1.2 Dijkstra's Algorithm . . . 5
3.1.3 Greedy Best-First Search . . . 5

3.1.4 Per-Table Comparison . . . 5
3.2 DWA Controller (Path Following) . . . 5
3.2.1 Kinematic Model . . . 6
3.2.2 Dynamic Window . . . 6
3.2.3 Cost Function . . . 6

4 Reinforcement Learning Navigation **6**
4.1 Why Actor-Critic? . . . 6
4.2 PPO Architecture . . . 7
4.3 Observation Space (26 values = the "State") . . . 7
4.4 Action Space (2 continuous values) . . . 7
4.5 Reward Function . . . 7
4.6 PPO Loss Function . . . 8
4.7 Training Hyperparameters . . . 8
4.8 Training Results . . . 9

5 Comparative Analysis **9**
5.1 Benchmark Methodology . . . 9
5.2 Results . . . 10
5.3 Analysis . . . 10
5.3.1 Path Efficiency . . . 10
5.3.2 Success Rate . . . 11
5.3.3 Obstacle Avoidance . . . 11
5.3.4 Computation Time . . . 11
5.4 Resource Comparison . . . 11

6 Discussion **11**
6.1 Summary . . . 11
6.2 Potential Improvements . . . 11

7 Conclusion **12**

1 Introduction

1.1 Use Case

Use Case: Restaurant Robot Waiter

A restaurant wants to automate table service using a mobile robot. The robot must autonomously navigate from the kitchen to **5 different tables** in the dining area, avoiding walls, furniture, and obstacles along the way. The goal is to compare two approaches: classical algorithms vs. artificial intelligence.

1.2 Objectives

1. Build a simulated restaurant environment using Gazebo and ROS
2. Implement classical path planning (A*, Dijkstra, Greedy) with a DWA controller
3. Implement a reinforcement learning agent (PPO Actor-Critic)
4. Compare both paradigms on path efficiency, success rate, obstacle avoidance, and computation time

1.3 System Overview

The system uses a **TurtleBot3 Burger** simulated in Gazebo. The robot starts at position $(19.0, 9.0)$ in the kitchen and must reach one of 5 tables in the dining area.

2 Environment Setup

2.1 Restaurant World

The restaurant was modeled in Gazebo using an aerial photograph as reference. The `.world` file contains walls (box geometries), 5 tables (`cafe_table` models), and a kitchen area. Table positions are stored in a JSON configuration file.

2.2 Map Generation

The occupancy grid map was generated using **SLAM** (`slam_toolbox`). Initial attempts with `explore_lite` and `gmapping` failed due to wheel slippage. A Python script was used to capture the map reliably.

Map Parameters

- **Resolution**: 0.05 m/pixel | **Origin**: $(-2.0, -2.0)$
- **Occupied threshold**: 0.65 | **Free threshold**: 0.196
- **Format**: PGM (Portable Gray Map) + YAML metadata

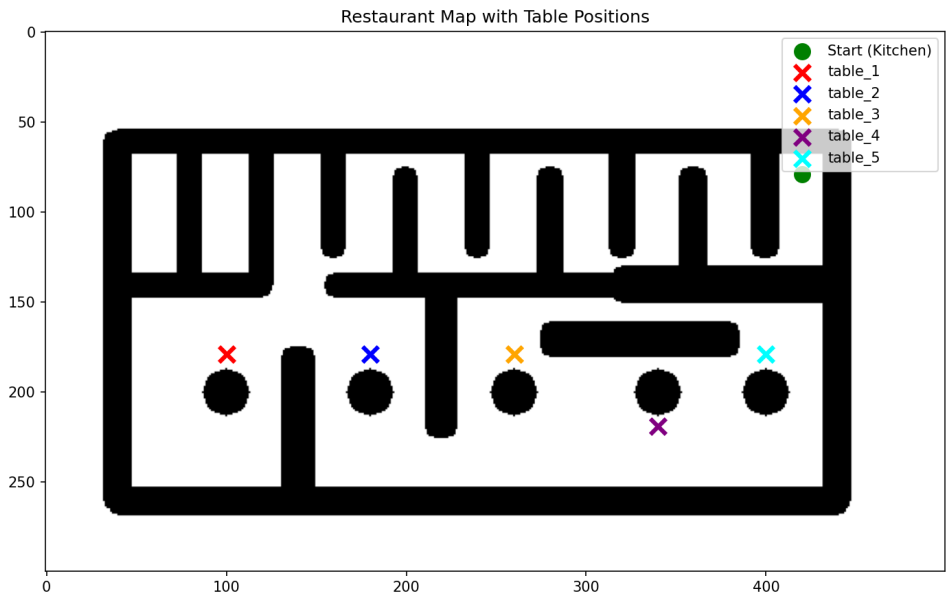

Figure 1: Restaurant occupancy grid with start position (green) and the 5 table delivery points.

2.3 Coordinate Transformation

Gazebo coordinates (meters) are converted to grid coordinates (pixels) using:

$$\text{col}_x = \text{round}\left(\frac{x_m - x_{\text{origin}}}{\text{resolution}}\right), \qquad \text{row}_y = (H-1) - \text{round}\left(\frac{y_m - y_{\text{origin}}}{\text{resolution}}\right) \tag{1}$$

2.4 Obstacle Inflation

To prevent collisions, obstacles are inflated (dilated) on the grid:

$$r_{\text{inflation}} = \left\lceil \frac{r_{\text{robot}} + r_{\text{margin}}}{\text{resolution}} \right\rceil = \left\lceil \frac{0.20 + 0.05}{0.05} \right\rceil = 5 \text{ pixels} \tag{2}$$

A circular structuring element is used with SciPy's `binary_dilation`.

3 Classical Navigation

3.1 Path Planning Algorithms

Three graph-search algorithms were implemented on the inflated 4-connected grid.

3.1.1 A* Algorithm

A* combines actual cost $g(n)$ with a heuristic $h(n)$:

$$f(n) = g(n) + h(n), \qquad h(n) = |x_n - x_g| + |y_n - y_g| \text{ (Manhattan)} \tag{3}$$

A* is **optimal** and **complete** when $h(n)$ is admissible.

3.1.2 Dijkstra's Algorithm

Dijkstra is A* with $h(n) = 0$:

$$f(n) = g(n) \tag{4}$$

It guarantees the shortest path but explores more nodes (no directional guidance).

3.1.3 Greedy Best-First Search

Greedy uses only the heuristic:

$$f(n) = h(n) \tag{5}$$

Fastest algorithm, but **not optimal** — the path may be longer than the shortest.

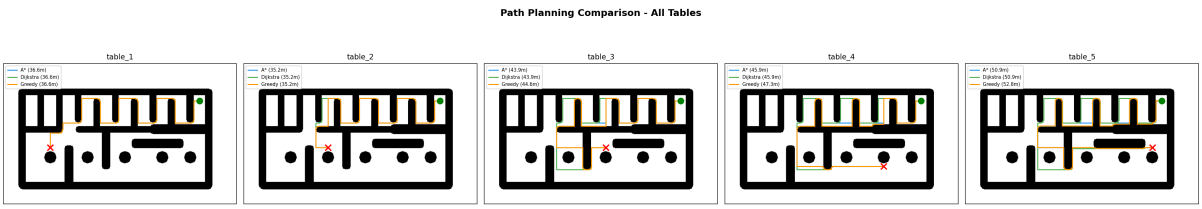

Figure 2: Path planning comparison for all 5 tables (A* in blue, Dijkstra in green, Greedy in orange).

3.1.4 Per-Table Comparison

Table 1: Path planning results for each table (path length in meters, explored nodes, computation time)

	A*			Dijkstra			Greedy		
Table	**m**	**nodes**	**ms**	**m**	**nodes**	**ms**	**m**	**nodes**	**ms**
table_1	36.6	10151	30	36.6	19887	45	36.6	4252	11
table_2	35.2	12135	49	35.2	17234	44	35.2	4654	9
table_3	43.9	19396	56	43.9	33614	77	44.6	10323	22
table_4	45.9	16688	48	45.9	37564	86	47.3	11149	29
table_5	50.9	19635	45	50.9	44719	96	52.8	12942	22
Average	**42.5**	**15601**	**46**	**42.5**	**30604**	**70**	**43.3**	**8664**	**18**

Key observations: A* and Dijkstra always find the **same optimal path**, but Dijkstra explores ∼2× more nodes. Greedy is 2.5× faster but produces paths 1–4% longer.

3.2 DWA Controller (Path Following)

The **Dynamic Window Approach** translates the grid path into velocity commands (v, ω) for the real robot in Gazebo.

3.2.1 Kinematic Model

$$\begin{cases} x_{t+1} = x_t + v\cos(\theta_t)\cdot\Delta t \\ y_{t+1} = y_t + v\sin(\theta_t)\cdot\Delta t \\ \theta_{t+1} = \theta_t + \omega\cdot\Delta t \end{cases} \qquad \text{with } \Delta t = 0.2 \text{ s} \tag{6}$$

3.2.2 Dynamic Window

Feasible velocities are constrained by acceleration limits:

$$V_s = \{(v,\omega) \mid v \in [v_c - a_v\Delta t,\ v_c + a_v\Delta t],\ \omega \in [\omega_c - a_\omega\Delta t,\ \omega_c + a_\omega\Delta t]\} \tag{7}$$

with $v_{\max} = 0.5$ m/s, $\omega_{\max} = 1.0$ rad/s, $a_v = 0.5$ m/s^2, $a_\omega = 1.0$ rad/s^2.

3.2.3 Cost Function

Each candidate trajectory is evaluated over a horizon of $H = 5$ steps:

$$J(v,\omega) = \underbrace{2.0\cdot d_{\text{path}}}_{\text{stay on path}} + \underbrace{1.0\cdot\Delta\theta_{\text{heading}}}_{\text{face the goal}} + \underbrace{0.5\cdot(v_{\max} - v)}_{\text{go fast}} \tag{8}$$

4 Reinforcement Learning Navigation

4.1 Why Actor-Critic?

Why PPO Actor-Critic instead of DQN or Q-Learning?

The TurtleBot3 is controlled with **continuous velocity commands**: linear velocity $v \in [0, 0.5]$ m/s and angular velocity $\omega \in [-1.0, 1.0]$ rad/s.
Classical RL methods like **DQN** (Deep Q-Network) only work with **discrete actions** (e.g., "turn left", "go forward"). This would force us to discretize the velocity space, losing precision and smooth control.
Actor-Critic methods (like PPO) natively support **continuous action spaces**:

- The **Actor** outputs a Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$ over velocities
- Actions are sampled continuously, bounded by the TurtleBot3's physical limits
- This produces **smooth, natural trajectories** instead of jerky discrete steps

4.2 PPO Architecture

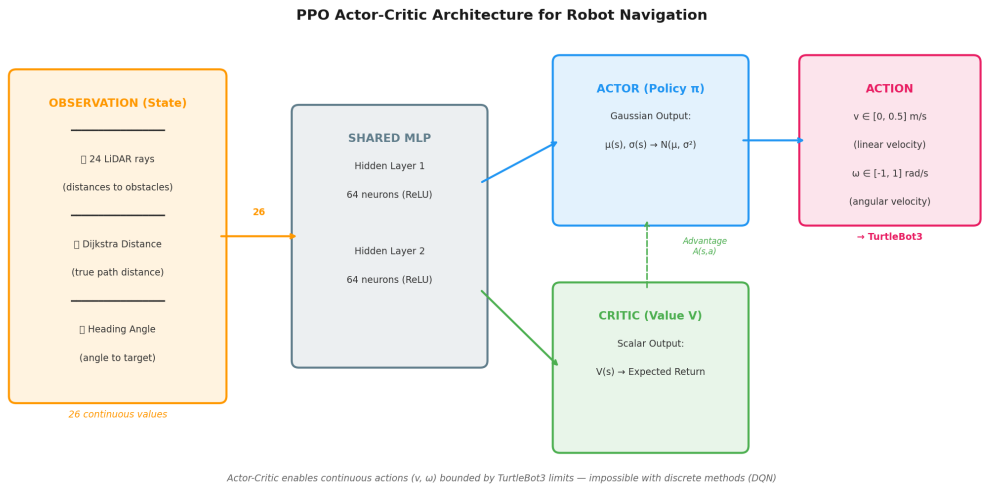

Figure 3: PPO Actor-Critic architecture. The shared MLP processes the 26-value observation, then splits into an Actor (continuous actions) and a Critic (value estimation).

4.3 Observation Space (26 values = the "State")

What the robot "sees" at each step (26 numbers)

The agent receives a vector of 26 continuous values:

1. **24 LiDAR rays** – Simulated laser scanner covering 360° around the robot (one ray every 15°). Each ray returns the **distance to the nearest obstacle** (0 to 3.5 m). This is how the robot "sees" walls and furniture.
2. **1 Dijkstra distance** – The **true shortest-path distance** to the target table, computed via a pre-calculated distance map (Dijkstra flood-fill). Unlike Euclidean distance, this accounts for walls and corridors.
3. **1 Heading angle** – The angle between the robot's current orientation and the direction to the goal ($\phi \in [-\pi, \pi]$). This tells the robot which way to turn.

4.4 Action Space (2 continuous values)

The Actor outputs two values, directly mapped to robot commands:

$$a = (v, \omega) \in [0.0, 0.5] \times [-1.0, 1.0] \tag{9}$$

These bounds match the TurtleBot3 Burger's physical velocity limits.

4.5 Reward Function

The reward guides the agent to reach the goal quickly and safely:

Reward Structure

$$R(s_t, a_t) = \begin{cases} +2000 & \text{if distance to table} < 0.35\text{m} \\ -500 & \text{if collision} \\ 50(D_{t-1} - D_t) - 0.5 - 0.3|\omega_t| & \text{otherwise} \end{cases}$$

Where the "otherwise" reward has three components:

- $50(D_{t-1} - D_t)$: **Progress reward** (Dijkstra distance)
- -0.5: **Time penalty** (per step)
- $-0.3|\omega_t|$: **Rotation penalty** (anti-zigzag)

- **Progress reward**: Uses the Dijkstra distance map (not Euclidean), so the robot is rewarded for navigating through corridors correctly.
- **Time penalty** (-0.5 per step): Forces the robot to find short paths, prevents infinite loops.
- **Rotation penalty** ($-0.3 \cdot |\omega|$): Penalizes spinning, produces smooth trajectories.

4.6 PPO Loss Function

PPO clips the policy update to prevent catastrophic changes:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right] \tag{10}$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio, $\hat{A}_t$ is the advantage, and $\epsilon = 0.2$.
Total loss:

$$L(\theta, \phi) = -L^{\text{CLIP}}(\theta) + 0.5 \cdot L^{\text{VF}}(\phi) - 0.01 \cdot H(\pi_\theta) \tag{11}$$

4.7 Training Hyperparameters

Table 2: PPO Training Configuration

Parameter	Value
Algorithm	PPO (Stable-Baselines3)
Network	MLP, 2 hidden layers $\times$ 64 neurons
Learning rate	1×10^{-4}
Rollout steps	4096
Batch size	128
Clip range (ϵ)	0.2
Entropy coefficient	0.01
Target KL divergence	0.015
Total timesteps	2,000,000
Episode timeout	1500 steps
Device	CUDA (GPU)

4.8 Training Results

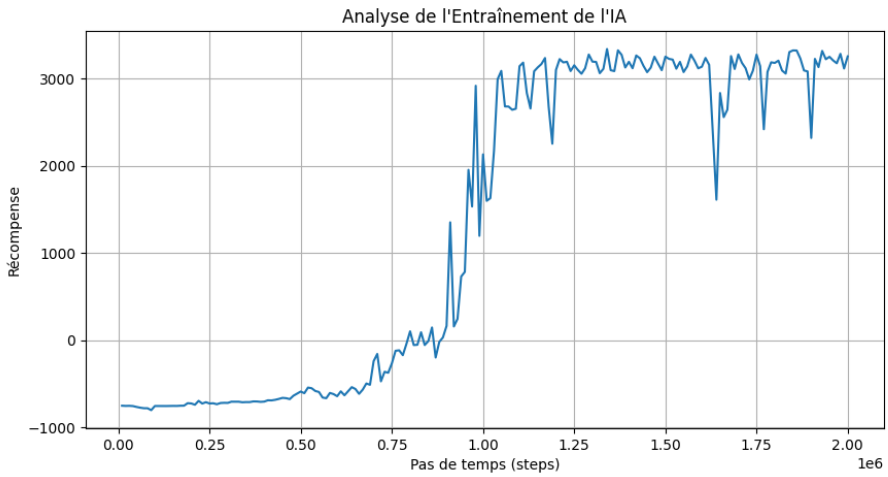

Figure 4: Training curve: mean evaluation reward over timesteps. The agent converges to stable positive rewards after ∼500k steps.

5 Comparative Analysis

5.1 Benchmark Methodology

Both paradigms were tested on all 5 tables from start position $(19.0, 9.0)$. Classical planners: 1 run per table (deterministic). RL agent: 5 trials per table (to measure variability).

5.2 Results

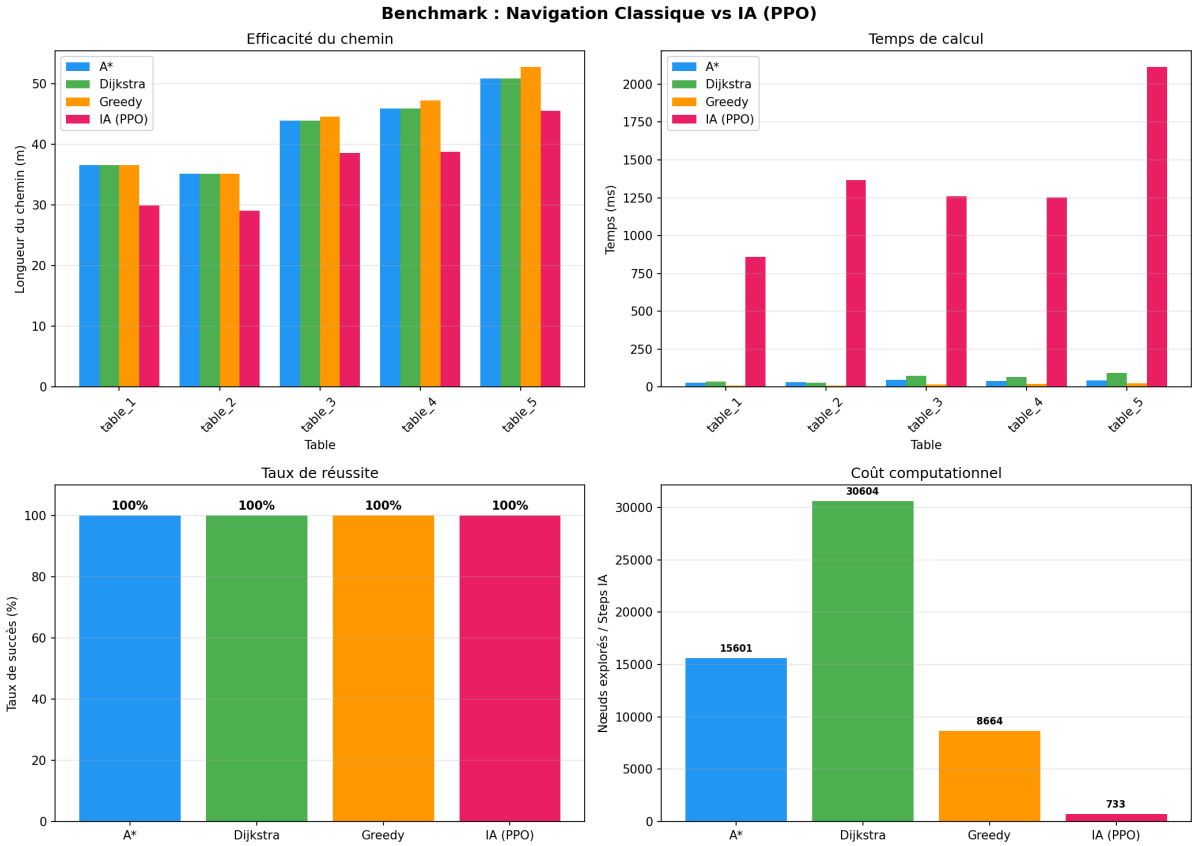

Figure 5: Comprehensive benchmark: path efficiency, computation time, success rate, and computational cost.

Table 3: Quantitative comparison (averaged over 5 tables)

Metric	A*	Dijkstra	Greedy	IA (PPO)
Success rate (%)	**100**	**100**	**100**	**100**
Mean path length (m)	42.50	42.50	43.30	**36.38**
Mean computation time (ms)	37.8	60.5	**15.5**	1371.4
Mean nodes / steps	15,601	30,604	8,664	**733**
Min. obstacle distance (m)	≥ 0.25	≥ 0.25	≥ 0.25	0.07
Crashes	0	0	0	0

5.3 Analysis

5.3.1 Path Efficiency

The RL agent produces **shorter paths** (36.38 m vs. 42.50 m), a **14.4% improvement**. This is because the RL agent moves in **continuous space** (smooth curves and diagonals), while grid planners are limited to 4-connected cardinal movements.

5.3.2 Success Rate

Classical planners achieve 100% (mathematically guaranteed). The RL agent achieves 100% after sufficient training (2M steps with target KL stabilization).

5.3.3 Obstacle Avoidance

Classical planners guarantee $\geq$ 25 cm clearance via obstacle inflation. The RL agent comes as close as 7 cm – safe in simulation but risky for real deployment.

5.3.4 Computation Time

Classical planners are **∼90× faster** at inference (15–60 ms vs. 1.4 s). However, the RL model requires no re-computation if the map changes – only re-training offline.

5.4 Resource Comparison

Table 4: Resource requirements

Resource	Classical (A* + DWA)	RL (PPO)
Training time	None	∼30 min (offline)
Inference time	∼40 ms	∼1.4 s
Memory	∼50 MB	∼200 MB
Dependencies	NumPy, SciPy	PyTorch, Gymnasium, SB3
GPU required?	No	Optional (faster)

6 Discussion

6.1 Summary

Table 5: Strengths and weaknesses

Criterion	Classical	RL (PPO)
Reliability	Deterministic, guaranteed	Stochastic, may fail
Path quality	Optimal on grid	Shorter (continuous)
Safety	Hard margins (inflation)	Learned, no guarantee
Adaptability	Needs re-planning	Adapts via learned policy
Setup effort	Simple algorithm choice	Training + reward engineering

6.2 Potential Improvements

- **Curriculum learning**: Start with nearby goals, progressively increase difficulty
- **Safety layer**: Override RL actions when obstacle distance $<$ 15 cm

- **Hybrid approach**: Use A* for global path + RL for local control (smooth following)
- **Domain randomization**: Add noise during training to improve sim-to-real transfer

7 Conclusion

Classical path planning (A* + DWA) provides a **reliable, fast, and safe** solution. Reinforcement learning (PPO Actor-Critic) produces **shorter, smoother paths** with continuous control, but requires training time and offers weaker safety guarantees.

For a production restaurant robot, a **hybrid approach** would be ideal: classical planning for guaranteed path safety combined with RL for smooth, natural robot motion.

References

[1] Schulman, J., et al. "Proximal Policy Optimization Algorithms." *arXiv:1707.06347*, 2017.

[2] Hart, P.E., Nilsson, N.J., Raphael, B. "A Formal Basis for the Heuristic Determination of Minimum Cost Paths." *IEEE Trans. SSC*, 1968.

[3] Fox, D., Burgard, W., Thrun, S. "The Dynamic Window Approach to Collision Avoidance." *IEEE Robotics & Automation Magazine*, 1997.

[4] Raffin, A., et al. "Stable-Baselines3: Reliable Reinforcement Learning Implementations." *JMLR*, 2021.

[5] ROBOTIS. *TurtleBot3 e-Manual*. https://emanual.robotis.com/docs/en/platform/turtlebot3/